



Bubble Sort 2 (Solution)

Adding $(\text{small value}) \times i$ to the i -term of the sequence, we may assume all the values of the sequence are different from each other.

For each element in the sequence, consider the set { elements which are prior to it and larger than it }. If this set is non-empty, after a pass, the element will move one step forward. Therefore, the maximum value of { elements which are prior to it and larger than it } will be the number of passes by bubble sort. They can take the maximum value only at vertices in the right lower convex hull of the sequence. For these vertices, the equality

$$\begin{aligned} & (\text{the number of elements which are prior to it and larger than it}) \\ &= i - (\text{the number of elements smaller than } A_i) \end{aligned}$$

holds. For other vertices, the inequality

$$\begin{aligned} & (\text{the number of elements which are prior to it and larger than it}) \\ &> i - (\text{the number of elements smaller than } A_i) \end{aligned}$$

holds. Therefore, the number of passes by bubble sort for the sequence A is calculated as

$$\max \left\{ i - (\text{the number of elements smaller than } A_i) \mid i = 0, 1, \dots, N-1 \right\}.$$

It is enough to keep the value of

$$i - (\text{the number of elements smaller than } A_i)$$

for each i , and to update and calculate the maximum value efficiently. Since the term i does not change, we may calculate it simply for each query. In order to update the term

$$-(\text{the number of elements smaller than } A_i),$$

for each query, we may add $+1$ or -1 to every element in the segment containing A_i .

Using Segment Tree, the above operations can be done $O(\log(N+Q))$ time per query. It takes $O((N+Q)\log(N+Q))$ time to compress the coordinates which appear in the sequence. Therefore, in total, this problem can be solved in $O((N+Q)\log(N+Q))$ time.



Cats or Dogs (Solution)

Let us restate the problem in terms of graphs. Then the garden becomes a tree structure, and each hut becomes a vertex. For simplicity, a hut with a cat is called a “red” vertex, and a hut with a dog is called a “blue” vertex.

Subtask 1

The tree has small number of vertices only. When we calculate the danger level, we can try all possible ways to cut the edges. Once we decide which edges we will cut, it is easy to calculate the danger level. For each query, the time complexity is $O(N \cdot 2^{N-1})$.

Subtask 2

For each query, it is enough to solve the problem in linear time. In the above solution, we did not use the tree structure. Let us consider the solution using the tree structure.

Let us consider the solution using Dynamic Programming on the tree. We choose a vertex as a root, and consider the graph as a rooted tree. For each vertex v , let R_v be the minimum number of edges we have to cut if v belongs to the “red connected component.” Similarly, let B_v be the minimum number of edges we have to cut if v belongs to the “blue connected component.” We shall update these values from the leaves of the tree. Here, “red connected component” is a connected component without blue vertices after we cut the edges. The definition of “blue connected component” is similar.

We have the following recurrence formula:

$$\begin{aligned} \bullet \quad B_v &= \sum_{c: \text{a child of } v} \min(B_c, R_c + 1) \\ \bullet \quad R_v &= \sum_{c: \text{a child of } v} \min(R_c, B_c + 1) \end{aligned}$$

We can calculate them using Depth-First Search. For each query, we can solve the problem in $O(N)$ time. The total time complexity is $O(NQ)$.

Subtask 3 (Full Score)

What are the properties we did not use in the above solutions? We do not use the fact that the status of at most one vertex will change for each day.

Let us try to optimize the above recurrence formula. We may consider using Heavy Light Decomposition. But, we will not explain this technique in detail because many documents are available on the web. In the following, when we use HLD, we shall explain what kind of data structures we need to consider.



The problem may be reduced to the case of the straight line. But, if we consider the original problem when the graph is the straight line, we can solve it easily. So, let us consider the situation where the values of B_v , R_v are fixed for each vertex. Then the problem may be reduced to this case appropriately.

By this reduction, the problem becomes as follows.

- In a sequence, there are N cells. Each cell will be colored red or blue.
- For each cell, the cost is R_i if it is colored red, and the cost is B_i if it is colored blue.
- However, if there are two adjacent cells with different colors, the cost is 1 for each of such pairs.
- Minimize the total cost.

Reformulating the problem in this way, you may think whether it can be solved using Segment Tree. In fact, for each interval, it is enough to keep the minimum of the total cost if we fix the colors (red or blue) for the two extreme vertices.

By this method, we can update each interval in $O(\log N)$ time. In total, for each query, we can solve the problem in $O(\log^2 N)$ time.



Collapse (Solution)

Subtask 1

Simulate the plan using Union Find.

Subtask 2

Since the place of collapse is fixed, we can consider the left side and the right side of it independently. Using Segment Tree, we keep the information of cities which are connected for consecutive period. Using reversible Union Find and applying Euler Tour to Segment Tree, we can calculate the number of connected components for each time.

Subtask 3

Apply the Square Root Decomposition to the number of cities in the upstream side and the downstream side. For the operations for each block, using one Union Find, we handle the cables in the block for which collapse does not happen. For the cables for which collapse happens, using the information of the previous Union Find, we can calculate the number of connected components using another Union Find.

Subtask 4 (Full Score)

Apply the Square Root Decomposition to the queries. For the operations for each block, sort the queries according to the positions where the collapses happen. We handle the cables in the block where constructions will not occur using one Union Find. For the cables in the block where constructions will occur, using the information of the previous Union Find, we can calculate the number of connected components using another Union Find.



Xylophone (Solution)

Subtask 1

In this subtask, we can send N^2 queries. So we can send queries for all the intervals.

First, we consider an interval for which the value at one of the ends of the interval is the maximum or minimum. We call such an interval a **good interval**. Since the situation is symmetric, in the following, we only consider an interval $A[M] \sim A[M + K - 1]$ whose leftmost value $A[M]$ is the minimum of the interval. The return value $\text{Query}(M, M + K - 1)$ is the difference of the maximum and the minimum of the interval. We denote it by D . If we ask $\text{Query}(M, M + K - 1)$, $\text{Query}(M, M + K - 2)$, $\dots$ in this order, then the return values are non-increasing. For the smallest integer i with $\text{Query}(M, M + K - i) \neq D$, the value $A[M + K - i + 1]$ is the maximum of the interval, and we know this value. Then, both of the interval $A[M] \sim A[M + K - i]$ and the interval $A[M + K - i + 1] \sim A[M + K - 1]$ are good intervals. Repeating this process, we can calculate all the values in a good interval.

It is enough to divide the initial sequence into two good intervals. If we ask $\text{Query}(1, N)$, $\text{Query}(1, N - 1)$, $\dots$ in this order, then, for the smallest integer i with $\text{Query}(1, i) \neq N - 1$, the integer $A[i + 1]$ is the maximum. Then, both of the interval $A[1] \sim A[i + 1]$ and the interval $A[i + 1] \sim A[N]$ are good intervals.

Subtask 2

In the solution of Subtask 1, we can do binary search instead of “asking in this order.” Then we can calculate one value by $\log N$ queries. Therefore, in total, we can solve this task by $N \log N$ queries.

Subtask 3 (Full Score)

We send queries for all the adjacent pairs ($N - 1$ pairs) and all the adjacent triples ($N - 2$ triples).

For every triple $A[i], A[i + 1], A[i + 2]$,

- if $\text{Query}(i, i + 2) = \text{Query}(i, i + 1) + \text{Query}(i + 1, i + 2)$ holds, then the relation between $A[i]$ and $A[i + 1]$ (increasing/decreasing) and the relation between $A[i + 1]$ and $A[i + 2]$ (increasing/decreasing) are the same, and
- if $\text{Query}(i, i + 2) \neq \text{Query}(i, i + 1) + \text{Query}(i + 1, i + 2)$ holds, then the relation between $A[i]$ and $A[i + 1]$ (increasing/decreasing) and the relation between $A[i + 1]$ and $A[i + 2]$ (increasing/decreasing) are different.

We know the difference of two adjacent numbers since we already asked for them. Adding differences appropriately (according to increasing/decreasing relation), we know all the relative values of numbers.

Finally, looking at the maximum and the minimum, we change the signs if necessary, and add adequate constants.